Machine Learning Laboratory — Naive Bayes Classification

Department of Computer Science & Engineering

Course Title: Machine Learning · Course Code: UE24CS352A · Semester: V

Academic Session: Aug–Dec 2026

---

1. Overview

This lab requires you to implement a **Multinomial Naive Bayes classifier from scratch in NumPy**, verify it against a reference implementation, and then use it to investigate how smoothing strength, feature representation and model combination affect classification performance.

The task is **sentence role classification**: given one sentence from the abstract of a randomised controlled trial, predict which section it came from — BACKGROUND, OBJECTIVE, METHODS, RESULTS or CONCLUSIONS. This is a real problem in biomedical text mining, where structuring unlabelled abstracts makes a literature corpus searchable by rhetorical role.

The lab has three parts. Part A builds and verifies the classifier. Part B explores smoothing and feature representation. Part C approximates the **Bayes Optimal Classifier** by combining several models weighted by their posterior probability.

---

2. Core Concepts & Mathematical Formulations

2.1 Bayes' rule and the classification decision

Naive Bayes assigns a document to the class with the highest posterior probability. By Bayes' rule:

$$P(c \mid x) = \frac{P(c)\,P(x \mid c)}{P(x)}$$

The evidence term $P(x)$ is identical for every class, so it cannot change which class wins and is dropped. The decision rule becomes:

$$\hat{c} = \underset{c}{\operatorname{argmax}}\; P(c)\,P(x \mid c)$$

$P(c)$ is the **prior** — how common the class is, before any words are examined. $P(x \mid c)$ is the **likelihood** — how probable this particular sentence is if it came from class c.

2.2 The multinomial model and the independence assumption

Modelling $P(x \mid c)$ for a whole sentence directly is impossible: there are vastly more possible sentences than training examples, so almost every sentence would have an estimated probability of zero. Naive Bayes sidesteps this by assuming the words are **conditionally independent given the class**:

$$P(x \mid c) = \prod_i P(w_i \mid c)^{x_i}$$

where x_i is the number of times word i occurs in the sentence. The decision rule is then:

$$\hat{c} = \operatorname*{argmax}_c P(c) \prod_i P(w_i \mid c)^{x_i}$$

This assumption is clearly false for natural language — "randomized" and "controlled" are not independent, nor are "confidence" and "interval". The assumption is what makes the model tractable, and one of the reflection questions asks you to work out what the model gets wrong because of it, and why it nevertheless classifies well.

2.3 Working in log space

Multiplying many small probabilities underflows. A 25-word sentence multiplies 25 values each perhaps around 10^{-5}, giving roughly 10^{-125} — which rounds to exactly 0.0 in double-precision floating point, leaving every class tied at zero and the prediction arbitrary.

Taking logarithms converts the product into a sum, which is numerically stable, faster to compute, and monotone — so it does not change which class wins:

$$\text{score}(c) = \log P(c) + \sum_i x_i \log P(w_i \mid c)$$

Your implementation must store and work with $\log P(c)$ and $\log P(w_i \mid c)$ throughout. The whole scoring step for an entire dataset is then a single matrix multiplication.

2.4 Estimating the parameters

Class prior. The maximum-likelihood estimate is the fraction of training *documents* belonging to that class:

$$P(c) = \frac{N_c}{N}$$

where N_c is the number of training sentences in class c and N the total. Note this counts documents, not words.

Word likelihood. The maximum-likelihood estimate is the share of class c's total word occurrences taken by word i:

$$P(w_i \mid c) = \frac{\text{count}(w_i, c)}{\sum_j \text{count}(w_j, c)}$$

2.5 Laplace (additive) smoothing

The estimate above has a fatal flaw. If word i never occurs in class c in the training data, $P(w_i \mid c) = 0$, which makes the entire product zero — in log space, $\log 0 = -\infty$. A single unseen word therefore eliminates a class outright, regardless of how strongly every other word in the sentence supports it. Since no training corpus contains every word in every class, this happens constantly.

Additive smoothing adds a pseudo-count α to every word in every class:

$$P(w_i \mid c) = \frac{\text{count}(w_i, c) + \alpha}{\sum_j \text{count}\left(w_j, c\right) + \alpha V}$$

where V is the vocabulary size. Every word now has non-zero probability in every class.

The αV term in the denominator is essential and easy to omit: there are V words, each receiving α added mass, so the denominator must grow by αV for the distribution to sum to one. Without it the result is not a probability distribution at all.

$\alpha = 1$ is called **Laplace smoothing**; smaller values are often called **Lidstone smoothing**. Section 6 asks you to tune α and explain the trade-off it controls.

2.6 Feature representation: counts and TF-IDF

Count features record how many times each vocabulary word occurs in the sentence. This is the representation the multinomial model is derived for.

TF-IDF rescales each count by how rare the word is across the whole corpus:

$$\text{tfidf}(w, d) = \text{tf}(w, d) \times \log \frac{N}{\text{df}(w)}$$

where $\text{df}(w)$ is the number of documents containing w. Words appearing in nearly every document receive a weight near zero; words concentrated in a few documents keep their weight. The resulting vectors are usually L2-normalised, so their values are fractions rather than integers.

Note that TF-IDF values are not counts, so feeding them to a multinomial model steps outside the model's derivation. It is standard practice and often works well, but the justification becomes empirical rather than principled — and the change of scale interacts with α, which Section 7 asks you to investigate.

N-grams. An n-gram range of $(1,2)$ adds every adjacent word pair as its own feature, so "double blind" becomes a feature in its own right alongside "double" and "blind". This captures short phrases the independence assumption would otherwise destroy, at the cost of a much larger vocabulary.

2.7 The Bayes Optimal Classifier

Everything above selects a single hypothesis h and classifies with it. The **Bayes Optimal Classifier (BOC)** does not commit to one. It averages the predictions of every hypothesis in the hypothesis space H, each weighted by how probable that hypothesis is given the training data:

$$P(c \mid x, D) = \sum_{h \in H} P(c \mid x, h)\, P(h \mid D)$$

No other classifier using the same hypothesis space and the same prior can achieve a lower expected error — this is what "optimal" means here, and it is a theorem, not a claim about any particular dataset.

The BOC is not computable in practice, because H is infinite. What you will build is an approximation: a small set of trained models standing in for H, with each one's posterior weight estimated empirically and their predicted probabilities combined by **soft voting** (a weighted average of the predicted class probabilities, as opposed to hard voting, which counts only each model's single predicted label).

Estimating $P(h \mid D)$. With a uniform prior over the candidate models, the posterior is proportional to the likelihood of held-out data under each model:

$$P(h \mid D) \propto P(D_{\text{held}} \mid h) = \prod_{k} P(y_k \mid x_k, h)$$

In log space this is a **sum** of per-example log-likelihoods. Section 8 asks you to compute the weights two ways — from the mean log-likelihood and from the summed log-likelihood — and to account for the difference between them.

---

3. Implementation Guide

Work through the notebook in order. Several TODOs name an **exact variable or attribute** because later cells depend on it.

Before you begin, upload train.txt, dev.txt and test.txt from the PubMed 20k RCT dataset to the same directory as the notebook.

Part A — Building and Verifying the Classifier

Sections 1–2: Setup and data loading.*(Fully worked — run as-is.)* Enter your SRN; its last three digits set the Part C subsample size. The dataset ships pre-split into three files, so no splitting is required. Examine the printed class distribution before continuing — it is the subject of the first reflection question.

Section 3: The classifier. Implement the NaiveBayesClassifier class, completing five TODOs across three methods:

- fit — compute the log prior and the smoothed log likelihood for each class, then store them as self.class_log_prior_ (shape (n_classes,)) and self.feature_log_prob_ (shape (n_classes, vocab_size)). Both arrays must be ordered to match self.classes_.
- predict_log_scores — return the un-normalised log posterior for every class as a single matrix operation. No loop over documents.
- predict — take the argmax across the class axis and convert the resulting index into a class label via self.classes_.

Those two attribute names are deliberately the ones scikit-learn uses, which is what makes the next section's check a direct comparison.

Section 4: Feature extraction and verification. Build count features with CountVectorizer, fitted on the **training split only**. Then run the provided verification cell.

This cell compares your learned parameters against scikit-learn's MultinomialNB, which computes the same quantities with the same smoothing. If your implementation is correct, the two must agree to floating-point precision. **Your maximum difference must be below 10^{-10} for both the priors and the likelihoods.**

Run this before looking at any accuracy score. The reason is that **a wrong Naive Bayes still classifies**: several plausible implementation errors shift every class score by a constant, leaving the argmax — and therefore the accuracy — almost unchanged, while the model's outputs stop being probabilities. Such a model trains, predicts, and produces a sensible-looking confusion matrix. A reference comparison tests the numbers themselves rather than the decisions they lead to, and either passes or fails.

Section 5: Baseline evaluation. Train at $\alpha = 1.0$, score on the test set, print the classification report and plot the row-normalised confusion matrix. Each row of that matrix shows where the sentences of one true class were sent, so the off-diagonal entries identify which classes the model confuses — information no single accuracy number contains.

Part B — Smoothing and Features

Section 6 — Experiment 1: Smoothing strength. Sweep α over $[0.01, 0.1, 0.5, 1.0, 2.0]$, evaluating each on the **dev** set with macro-F1. Store the results in alpha_scores, select best_alpha, and plot the curve on a log-scaled x-axis. Only after the value is fixed should you refit and score on the test set.

Section 7 — Experiment 2: Counts versus TF-IDF. Build a Pipeline combining TfidfVectorizer with MultinomialNB and evaluate it with default settings. Then tune both the n-gram range and α together using GridSearchCV with 3-fold cross-validation on the dev set, scoring by macro-F1. Refit the winning configuration on the full training set and score on test.

From this section onward, use scikit-learn's MultinomialNB rather than your own class — Section 4 established they are identical, and the pipeline machinery is the subject here.

Part C — The Bayes Optimal Classifier

Section 8: Posterior-weighted combination. Four hypotheses are provided (Naive Bayes, Logistic Regression, Decision Tree, Random Forest), all trained on a subsample of the training data whose size comes from your SRN.

1. Split the subsample into a fitting part and a held-out part.
2. Fit each hypothesis on the fitting part and compute its **mean log-likelihood** of the true labels on the held-out part. Clip the predicted probabilities away from zero first — a single confident mistake otherwise produces $\log 0 = -\infty$ and destroys that model's weight.
3. Convert the mean log-likelihoods into weights with a softmax, subtracting the maximum before exponentiating to avoid overflow. Store them as posterior_weights.
4. Also compute the weights from the **summed** log-likelihood, which is the exact Bayesian quantity, and compare the two sets.

5. Fit a VotingClassifier with voting="soft" and those weights on the full subsample, score it on the test set, and then score each individual hypothesis on the test set for comparison.

Step 5's individual scores are not optional — the reflection question depends on comparing the ensemble against each of its members.

Sections 9–10: Results and reflection. Assemble the logged metrics into a summary table and answer the four synthesis questions.

---

4. Diagnostic & Evaluation Standards

4.1 Evaluation metrics

For a single class, writing TP, FP and FN for true positives, false positives and false negatives:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Macro-averaged F1 is the unweighted mean of the per-class F_1 scores:

$$F_1^{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} F_1^{(c)}$$

Because it is unweighted, every class contributes equally regardless of size, so a model that performs badly on a rare class cannot hide behind its performance on common ones. This is the primary metric for this lab. Report accuracy alongside it, but draw your conclusions from macro-F1 and say so.

4.2 Reading a confusion matrix

Row-normalising the confusion matrix — dividing each row by its total — makes each row the distribution of predictions for one true class. The diagonal entry is that class's recall, and the off-diagonal entries show exactly where its errors went. Two classes that are frequently exchanged indicate overlapping vocabulary rather than a generally weak model, and that distinction is invisible in any single-number summary.

4.3 Protocol integrity

Split	Purpose
Train	Fits the model — the word counts and the priors.
Dev	Chooses hyperparameters (α, n-gram range, model selection).
Test	Scored, after every choice has been fixed.

The reason for the separation: any data used to make a choice has participated in fitting the model. A score computed on data that selected a hyperparameter is the maximum over the options tried, not an estimate of performance on unseen data, and it is optimistically biased by an amount that cannot be measured from inside the experiment. A large test set reduces the variance of each individual estimate but does nothing about this bias, which arises from the selection step rather than from noise.

5. Submission Checklist & Deliverables

Jupyter Notebook (.ipynb)

- ☐STUDENT_ID set to your own SRN
- ☐ All # TODO cells implemented and producing output
- ☐**The Section 4 verification cell passing** (maximum difference $< 10^{-10}$)
- ☐predict_log_scores implemented without a Python loop over documents
- ☐ Every reflection question answered in your own words (3–5 sentences unless stated otherwise)
- ☐ All cells run in order, top to bottom, with outputs visible

A notebook that only works when cells are run out of order will not be accepted.

Lab Report (.pdf)

Title page — project title, your name, SRN, course code (UE24CS352A), date.

1. Introduction — the classification task and the purpose of the lab.

2. Dataset description — the three splits and their sizes, the five classes and their distribution, your Part C subsample size, and the vocabulary size after vectorisation.

3. Methodology — the mathematical formulation of the multinomial Naive Bayes model, your smoothing scheme, and how you verified your implementation.

4. Results and analysis — the complete results table (accuracy and macro-F1 for every experiment), the α sweep curve, confusion matrices for at least the baseline and your best model, and an explicit analysis of which classes are confused and why.

5. Conclusion — your findings on smoothing, feature representation, and whether the posterior-weighted ensemble improved on its individual members.